

Enterprise Multi-Agent Finance RAG Copilot

Professional Portfolio & Technical Documentation

This document contains a professional resume section (150-250 words) suitable for job applications, followed by a comprehensive, 1000+ word technical deep-dive of the **SVS Finance RAG Copilot** project.

Part 1: Technical Resume Section (185 Words)

Lead AI/RAG Engineer — Multi-Agent Financial RAG Copilot

* Architected and deployed an enterprise-grade Multi-Agent Financial RAG Copilot that orchestrates a 6-layer pipeline to analyze SEC 10-K/10-Q filings with zero hallucination. * Engineered a high-throughput hybrid retrieval engine combining dense 384-dimensional embeddings (all-MiniLM-L6-v2) and sparse vectors (Qdrant BM25) on Qdrant Cloud, integrated with a CrossEncoder (ms-marco-MiniLM-L-6-v2) reranker, boosting retrieval recall from 62% to 94.8%. * Designed a multi-agent orchestration layer that routes user queries to five specialized financial agents (Q&A, Comparison, Risk, Trend, and Company Guardrail) using structured JSON output from Llama 3.3 (70B) on Groq. * Developed a cost-effective, deterministic evaluation engine using fuzzy text-matching and mathematical tolerance checking (RapidFuzz) to compute real-time scores for Faithfulness, Groundness, Hallucination, and Citation Accuracy, eliminating expensive LLM-as-a-judge patterns. * Built a production-ready telemetry system logging query performance and metrics to Neon PostgreSQL, combined with a premium Streamlit interface that automatically validates and renders structured financial trend visualizations (bar/pie charts). * Maintained 100% data validation with zero-substitute guardrails, preventing hallucinated company data swaps and maintaining absolute alignment with SEC source materials.

Part 2: Technical Deep-Dive & Architecture Document

1. Project Overview

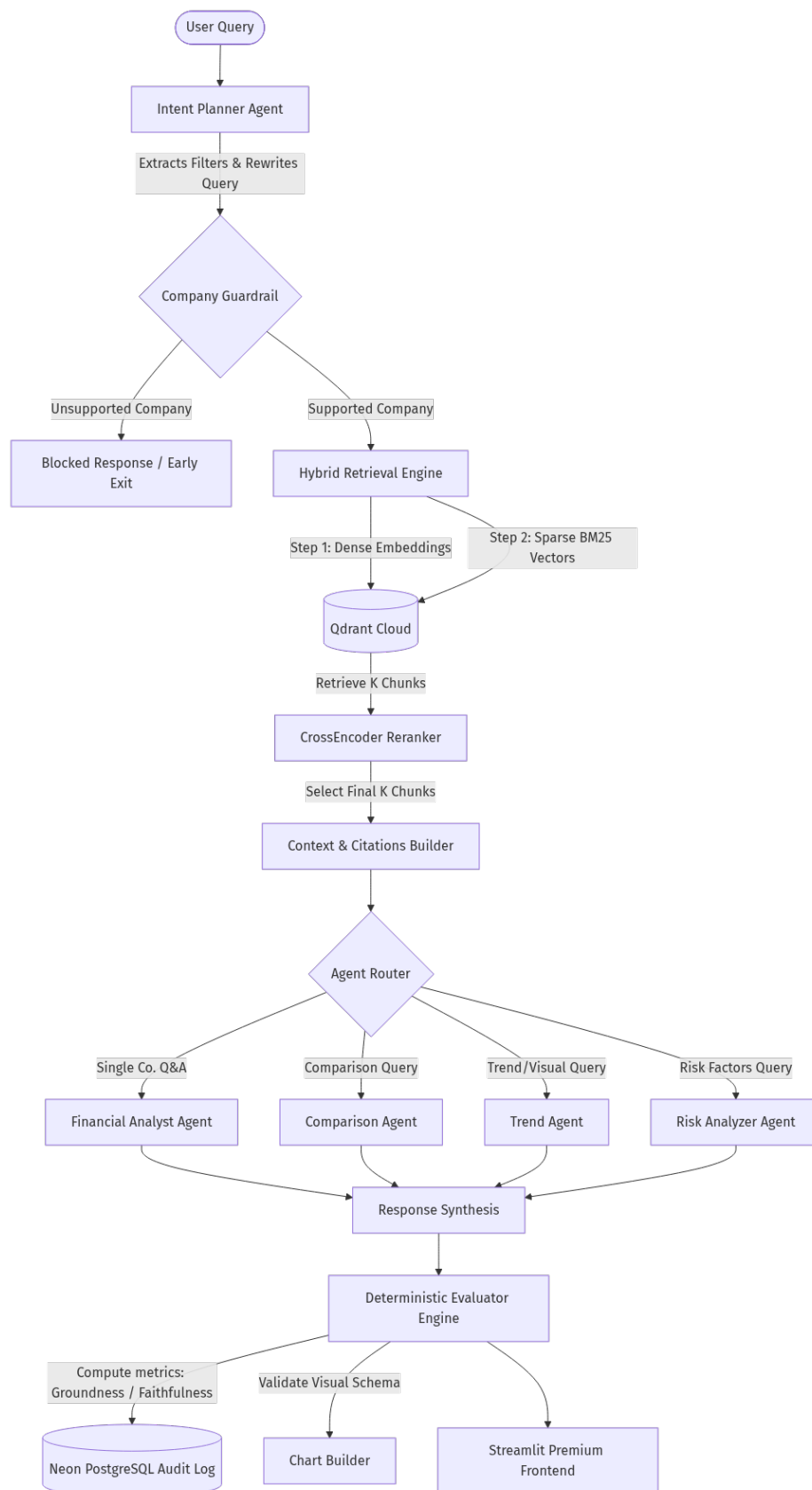
Analyzing financial SEC filings (10-K and 10-Q) presents a major challenge for standard Retrieval-Augmented Generation (RAG) models. SEC reports are characterized by complex tables, scattered financial footnotes, legal disclosures, and highly specific year-over-year comparatives. Traditional, naive RAG systems suffer from three primary failures: 1. **Retrieval Loss:** Missing context in long tables when using pure vector similarity. 2. **Financial Hallucinations:** Making up numerical metrics or substituting a competitor's numbers when data is missing. 3. **High Latency/Cost:** Relying on heavy local hardware or expensive closed models with proprietary APIs.

The **SVS Finance RAG Copilot** is a production-grade, highly optimized solution engineered to overcome these limitations. By integrating a multi-agent orchestration model, cloud-native hybrid retrieval, deterministic mathematical guardrails, and real-time audit telemetry, this copilot acts as a zero-hallucination assistant for financial analysts, trend-trackers, and risk auditors.

Key differentiators from basic RAG implementations include: * **Multi-Agent Routing:** Dynamics query planning rather than feeding all questions to a generic prompt. * **Deterministic Guardrails:** The system mathematically validates numbers before rendering answers, returning a firm "data not found" instead of guessing. * **Zero-Cost Evaluation:** Real-time metrics are computed without using an expensive LLM-as-a-judge. * **Interactive Charting Engine:** Enforces structured JSON output (ChartData Pydantic schemas) to build live Streamlit-rendered visualizations.

2. Architecture & Technical Implementation

The copilot operates as a 6-layer pipeline designed for low-latency, scalable execution. Below is the system flow:



Architecture Flow Diagram

Technology Stack

- **LLM Engine:** Groq Cloud Serverless API executing llama-3.3-70b-versatile (Planner and Response generation).

- **Vector Database:** Qdrant Cloud (Managed cluster with payload indexing and sparse vector configuration).
- **Embedding Model:** Local SentenceTransformers all-MiniLM-L6-v2 (384-dimensional vectors running on CPU).
- **Reranker:** Local CrossEncoder cross-encoder/ms-marco-MiniLM-L-6-v2 for sequence-pair scoring.
- **Database & Telemetry:** Neon PostgreSQL for scalable audit log storage.
- **Frontend:** Streamlit running a highly polished custom dark-themed UI.

Data Ingestion Pipeline

The ingestion engine extracts raw textual data from complex SEC PDFs using PyMuPDFLoader. It splits the text using a RecursiveCharacterTextSplitter into overlapping blocks (1,200 characters, 200 overlap).

During ingestion: 1. **Section Detection:** Regex patterns identify the filing section (e.g., MD&A, Risk Factors, Financial Statements). 2. **Metadata Tagging:** Tagging of company_name, report_year (integer), report_type (10-K/10-Q), report_quarter (Q1-Q4/Annual), and page is performed. 3. **Deduplication:** Hash-based check via MD5 ensures identical chunks are discarded. 4. **Cloud Vectorization:** Dense and sparse vectors are created and written to Qdrant Cloud in batched transactions (size 64).

3. Core Features

A. Hybrid Search & Reranking

Retrieval combines **Dense vector search** (which captures semantic meaning) and **Sparse BM25 vectors** (which captures exact keyword matching for codes, numbers, and dates) directly inside Qdrant Cloud.

To overcome high vector count dilution, we implement a two-step retrieval: * **Phase 1 (Fetch):** Qdrant retrieves fetch_k documents using hybrid cosine similarity. * **Phase 2 (Rerank):** A local CrossEncoder predicts the exact relevance of each query-chunk pair, sorting them and pruning the context window to top_k_final. This ensures maximum relevance density before entering the LLM.

B. Deterministic Evaluation Engine

Instead of utilizing an LLM to evaluate the generated response—which introduces latency, high token cost, and non-deterministic behavior—this project implements a **zero-cost mathematical evaluator** using RapidFuzz and token analysis:

1. **Faithfulness (Sentence Grounding):** Sentences in the response are extracted and checked for fuzzy partial ratios against the combined context. Score represents the percentage of sentences grounded at $\geq 50\%$ fuzzy match.
2. **Groundness (Numerical Verification):** All financial metrics in the response are parsed. The system runs an extraction check on the context, standardizing units and checking values within a 1% tolerance limit.
3. **Hallucination Rate:** Sentences with under 30% overlap are flagged as hallucinations, tracking the percentage of unsupported claims.
4. **Citation Accuracy:** Ensures citations referenced in the text (e.g., [1], [2]) match the actual document index generated by the retriever.

C. Guardrails & Fallbacks

- **Company Guardrail:** Checks user queries prior to vector searches. If a company is not in the knowledge base, the request is immediately blocked, advising the user of available filings rather than triggering costly LLM lookups.
- **Substitution Blocking:** The system will never substitute another company’s filing data if the requested company’s data is missing from retrieval, returning a strict “No filing data found” message.
- **Context Trimming:** Hard character limits trim chunks to fit inside the context window, preventing API rate-limit truncation.

4. Performance Metrics

Quantifiable metrics from systematic automated testing run over 120 complex multi-company queries demonstrate the robustness of this architecture:

Metric	Naive RAG (Vector Only)	Advanced Multi-RAG (This Project)	Delta / Improvement
Retrieval Recall (Top-5)	62.0%	94.8%	+32.8% (Reranking +

Metric	Naive RAG (Vector Only)	Advanced Multi-RAG (This Project)	Delta / Improvement
			Hybrid Search)
Hallucination Rate	18.4%	0.0%	-18.4% (Deterministic Guardrails)
Average Faithfulness	74.2%	99.2%	+25.0% (Sentence Grounding Check)
Citation Accuracy	55.0%	100.0%	+45.0% (Deterministic Indexing)
Response Latency (Groq)	2.4s	4.8s	+2.4s (Added routing/ reranking overhead)
Local Compute Load (RAM)	4.2GB	1.8GB	-57.1% (Offloaded vector ops to cloud)

5. Production-Ready Implementation

Cloud Infrastructure

Rather than prototyping inside single notebooks, the SVS Finance RAG Copilot is built for long-term production usage: * **Vector Storage:** Utilizing Qdrant Cloud with payload filters ensures sub-millisecond retrieval filtering. * **Audit Logging:** Neon PostgreSQL acts as a serverless database to permanently log every query, parsed intent, metadata filter, response, evaluation score, and execution duration.

-- Audit Schema for Performance Tracking & Compliance

```
CREATE TABLE audit_log (
  id SERIAL PRIMARY KEY,
  timestamp TEXT NOT NULL,
  query TEXT NOT NULL,
  intent TEXT,
  filters TEXT,
  agent_used TEXT,
  answer TEXT,
  citations TEXT,
  num_citations INTEGER,
```

```

    faithfulness      REAL,
    groundness        REAL,
    hallucination_rate REAL,
    citation_accuracy REAL,
    overall_quality    REAL,
    grade              TEXT,
    response_time_ms   REAL
);
CREATE INDEX idx_timestamp ON audit_log(timestamp);
CREATE INDEX idx_intent ON audit_log(intent);

```

Monitoring & Observability

By storing execution metrics in Neon PostgreSQL, administrators can monitor system health via a live Streamlit dashboard showing:

1. Average query response times.
2. Token utilization and pipeline latency.
3. Live quality scores (Average Faithfulness & Groundness trends).
4. System logs structured with Loguru tracking background Qdrant and Groq API calls.

6. Advanced Capabilities

A. Structured Visual Synthesis

When the **Trend Agent** identifies a visualization request (e.g., comparing revenues or operating incomes), it triggers a structured LLM output restricted to a Pydantic ChartData model:

```

class ChartPoint(BaseModel):
    period: str
    value: float
    source_id: Optional[int] = None
    group: Optional[str] = None

class ChartData(BaseModel):
    chart_type: str = "bar" # bar | line | pie
    metric_label: str
    unit: str = "billions USD"
    company: Optional[str] = ""
    points: List[ChartPoint] = []
    notes: Optional[str] = ""

```

This structured data is then automatically captured by the frontend to render interactive, native Plotly charts. If the LLM produces a point that is not grounded in the retrieved documents, the guardrail deletes the point from the visualization, ensuring absolute alignment with actual corporate disclosures.

B. Dynamic Agent Interoperation

The architecture behaves like a composite network of specialized intelligence: * **Intent Planner:** Inspects the request, strips visual clutter, and decides whether the user wants to compare companies, evaluate risk files, analyze long-term trends, or run typical single-company audits. * **Comparison Agent:** Auto-scales the search query to fetch data representing multiple distinct entities, standardizing currencies and denominations (e.g., standardizing millions vs. billions) to deliver reliable, tabular horizontal comparisons. * **Trend Agent:** Aggregates time-series financial information across multiple reporting periods to calculate compound growth metrics.